Construire des bases de données NoSQL

Choisissez votre famille NoSQL

Qu’est-ce que le NoSQL ?

Le NoSQL propose une façon de gérer les données, sans reposer sur le paradigme relationnel, d'où le "**Not Only SQL**". Cette approche propose de relâcher certaines contraintes lourdes du relationnel pour favoriser la distribution (structure des données, langage d'interrogation ou la cohérence).

Dans un contexte de bases de données, il est préférable d'avoir un langage de haut niveau pour interroger les données plutôt que tout exprimer en Map/Reduce. Toutefois, avoir un langage de trop haut niveau comme SQL ne facilite pas la manipulation. Et c'est en ce sens que l'on peut parler de "Not Only SQL", d'autres solutions peuvent être proposées pour résoudre le problème de distribution. Ainsi, le NoSQL est à la fois une autre manière d'interroger les données, mais aussi de les stocker.

Les besoins de stockage et de manipulation dans le cadre d'une base de données sont variables et dépendent principalement de l'application que vous souhaitez intégrer.

Différentes familles de bases NoSQL existent : *Clé/Valeur, colonnes, documents, graphes*. Chacune de ces familles répond à des besoins très spécifiques que nous allons développer par la suite pour vous permettre de choisir **votre** solution NoSQL.

Les clés-valeurs

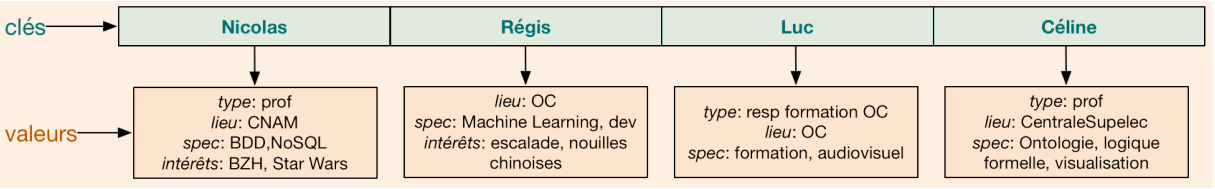
Le but de la famille clé-valeur est l'efficacité et la simplicité. Un système clé-valeur agit comme une énorme table de hachage distribuée sur le réseau. Tout repose sur le couple Clé/Valeur. La clé identifie la donnée de manière unique et permet de la gérer. La valeur contient n'importe quel type de données.

Le fait d'avoir n'importe quoi implique qu'il n'y ait ni schéma, ni structure pour le stockage. D'un point de vue de bases de données, il n'y a pas la possibilité d'exploiter ni de contrôler la structure des données et de fait, pas de langage (SQL = **Structured** Query Language). En soit ce n'est pas un problème si vous savez ce que vous cherchez (la clé) et que vous manipulez directement la valeur.

Les seules opérations de type CRUD peuvent être utilisées :

- Create (key,value)
- Read (key)

- Update (key,value)
- Delete (key)



Stockage Clé/Valeur

Qui s'intéresse à des fonctionnalités aussi simples ?

- **Redis** (*VMWare*) : Vodafone, Trip Advisor, Nokia, Samsung, Docker
- **Memcached** (*Danga*) : LiveJournal, Wikipédia, Flickr, Wordpress
- **Azure Cosmos DB** (*Microsoft*) : Real Madrid, Orange tribes, MSN, LG, Schneider Electric
- **SimpleDB** (*Amazon*)

Quels types d'applications pourraient correspondre à cette solution ? : *détection de fraude en temps réel, IoT, e-commerce, gestion de cache, transactions rapides, fichiers de logs, chat.*

Des lignes vers les colonnes

Traditionnellement, les données sont représentées en ligne Le stockage orienté colonne change ce paradigme en se focalisant sur chaque attribut et en les distribuant. Il est alors possible de focaliser les requêtes sur une ou plusieurs colonnes

Stockage orienté lignes

id	type	lieu	spec	intérêts
Nicolas	prof	CNAM	BDD, NoSQL	BZH, Star Wars
Régis		OC	Machine Learning, Dev	escalade, nouilles chinoises
Luc	resp formation OC	OC	formation, audiovisuel	
Céline	prof	CentraleSupelec	Ontologie, logique formelle, visualisation	

Stockage orienté colonnes

id	type
Nicolas	prof
Céline	prof
Luc	resp formation OC

id	lieu
Céline	Centrale Supelec
Nicolas	CNAM
Régis	OC
Luc	OC

id	spec
Nicolas	BDD
Nicolas	NoSQL
Régis	Machine Learning
Régis	Dev
Luc	formation
Luc	audiovisuel
Céline	Ontologie
Céline	logique formelle
Céline	visualisation

id	intérêts
Nicolas	BZH
Nicolas	Star Wars
Régis	escalade
Régis	nouilles chinoises

Stockage orienté colonnes

Cette solution est très adaptée pour effectuer des traitements sur des colonnes comme les agrégats (comptage, moyennes, co-occurences...). Elle est adaptée à de gros calculs analytiques. Toutefois, cette solution est moins appropriée pour la lecture de données spécifiques comme pour les clés/valeurs.

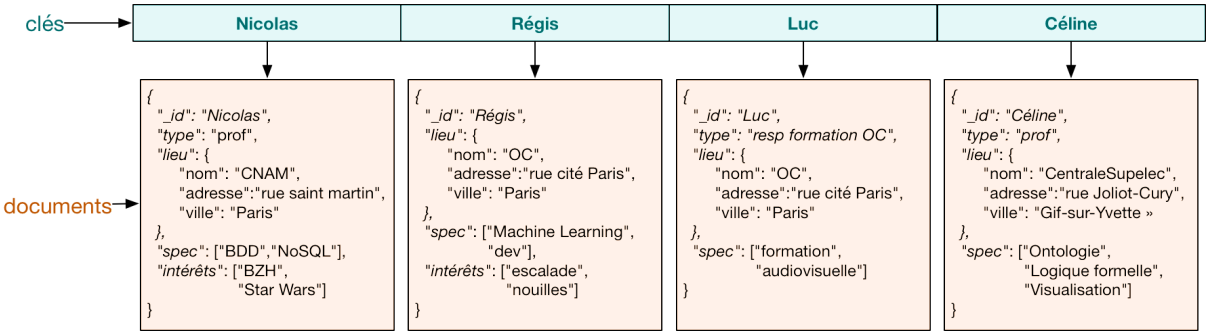
Qui fait de l'orienté-colonnes ?

- **BigTable** (*Google*)
- **HBase** (Apache, Hadoop)
- **Spark SQL** (*Apache*)
- **Elasticsearch** (*elastic*) -> moteur de recherche

Quelques exemples : *Comptage (vote en ligne, compteur, etc), journalisation, recherche de produits dans une catégorie, reporting à large échelle.*

La gestion de documents

Le but de ce stockage est de manipuler des documents contenant des informations avec une structure complexe (types, listes, imbrications). Il repose sur le principe du clé/valeur, mais avec une extension sur les champs qui composent ce document.



Stockage orienté document

L'avantage de cette solution est d'avoir une approche structurée de chaque valeur, formant ainsi un document. De fait, ces solutions proposent des langages d'interrogation riches permettant de faire des manipulations complexes sur chaque attribut du document (et sous-documents) comme dans une base de données traditionnelles, tout en passant à l'échelle dans un contexte distribué.

Quelles solutions pour gérer vos documents :

- **MongoDB** (*MongoDB*) : ADP, Adobe, Bosch, Cisco, eBay, Electronic Arts, Expedia, Foursquare

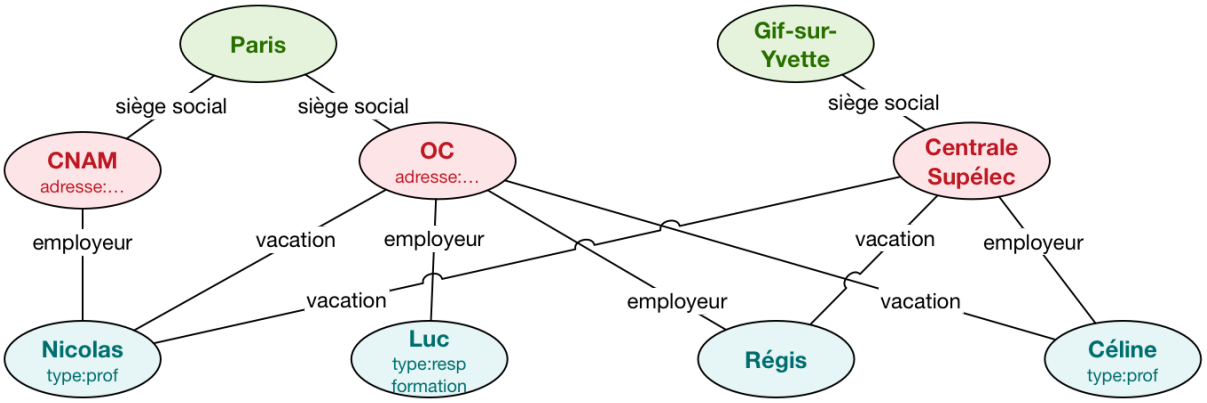
- **CouchBase** (*Apache, Hadoop*) : AOL, AT&T, Comcast, Disney, PayPal, Ryanair
- **DynamoDB** (Amazon) : BMW, Dropcam, Duolingo, Supercell, Zynga
- **Cassandra** (*Facebook -> Apache*) : NY Times, eBay, Sky, Pearson Education

Cassandra est souvent considérée comme une solution orientée colonnes. C'est une solution orientée documents appelée "wide-column store", dans le sens où le document est structuré comme en relationnel. La confusion vient du fait que l'on y définit des colonnes et des familles de colonnes dans le schéma, ce qui est différent du stockage de "colonnes".

Quelques exemples d'utilisation : *gestion de contenu (bibliothèques numériques, collections de produits, dépôts de logiciels, collections multimédia, etc.), framework stockant des objets, collection d'événements complexes, gestion des historiques d'utilisateurs sur réseaux sociaux.*

Et les graphes ?

Les trois premières familles NoSQL n'adressent pas le problème de corrélations entre les éléments. Prenons l'exemple d'un réseau social : dans certains cas, il devient très complexe de calculer la distance entre deux personnes non directement connectées. Et c'est ce type d'approche que résolvent les bases orientées Graphe.



Stockage orienté graphe

Dans la base orientée graphe, les données stockées sont : les nœuds, les liens et des propriétés sur ces nœuds et ces liens. Les requêtes que l'on peut exprimer sont basées sur la gestion de chemins, de propagations, d'agrégations, voire de recommandations.

Quelles bases de données gèrent de gros graphes ?

- **Neo4j** : eBay, Cisco, UBS, HP, TomTom, The National Geographic Society
- **OrientDB** (*Apache*) : Comcast, Warner Music Group, Cisco, Sky, United Nations, VErisign
- **FlockDB** (*Twitter*) : Twitter

Quelques idées d'applications reposant sur des bases orientées graphes : *réseaux sociaux (recommandation, plus court chemin, cluster...), réseaux SIG (routes, réseau électrique, fret...), web social (Linked Data).*

Le théorème de CAP

Propriétés fondamentales d'une base de données relationnelle

Le NoSQL est très tentant lorsque l'on voit toutes les possibilités qu'il peut offrir. Mais il ne faut pas négliger certains fondamentaux qui rendent les SGBDR incontournables.

Cela se résume dans une combinaison qu'il ne faut pas négliger :

1. Faire des jointures entre les tables de la base de données
2. Faire des requêtes complexes avec un langage de haut niveau sans se préoccuper des couches basses
3. Gérer l'intégrité des données de manière infaillible

Et c'est surtout ce dernier point qui attire l'attention dans le cadre du NoSQL. Gérer l'intégrité des données veut dire être capable de garantir le contenu de la base de données, quelles que soient les mises à jour effectuées sur les données, ainsi que sa pérennité. **ACID vs BASE**

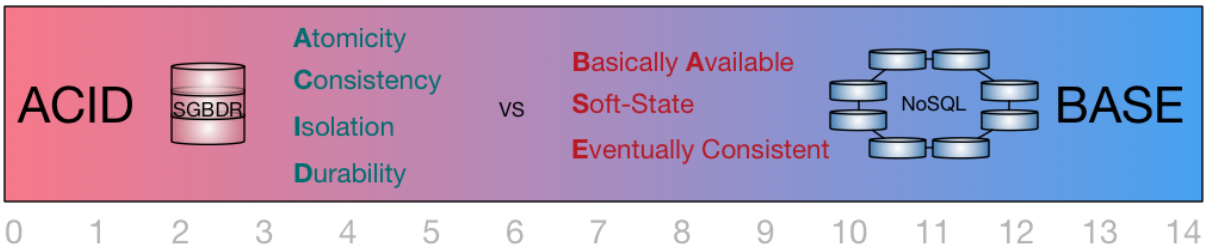
Pour mettre en rapport les problématiques de la base de données relationnelle, on parle de propriétés **ACID** pour les transactions (séquences d'opérations/requêtes) :

- **A***tomicité* : Une transaction s'effectue entièrement ou pas du tout
- **C***ohérence* : Le contenu d'une base doit être cohérent au début et à la fin d'une transaction
- **I***solation* : Les modifications d'une transaction ne sont visibles/modifiables que quand celle-ci a été validée
- **D***urabilité* : Une fois la transaction validée, l'état de la base est permanent

Toutefois, ces propriétés ne sont pas applicables dans un contexte distribué tel que le NoSQL. En effet, prenons l'exemple d'une transaction de cinq opérations (lecture/écriture) : cela implique une synchronisation entre cinq serveurs pour garantir l'atomicité, la cohérence et l'isolation. Cela se traduit par des latences dans

les transactions. Ce qui n'est pas tolérable lorsque l'on veut éviter ces latences en distribuant les calculs.

Si un serveur tombe en panne, il faut pouvoir garantir de retrouver toutes les données présentes sur ce serveur, donc on fait de la réplication. Il va falloir synchroniser toutes les mises à jour avec tous les réplicas de la donnée !



ACID vs BASE

Du coup, les propriétés **BASE** ont été proposées pour caractériser les bases NoSQL :

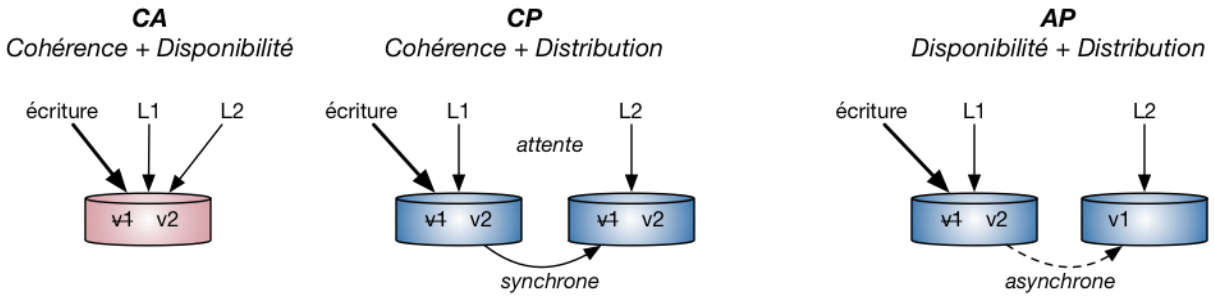
- **B***asically* ***A****vailable* : quelle que soit la charge de la base de données (données/requêtes), le système garantie un taux de disponibilité de la donnée
- **S***oft-state* : La base peut changer lors des mises à jour ou lors d'ajout/suppression de serveurs. La base NoSQL n'a pas à être cohérente à tout instant
- **E***ventually consistent* : À terme, la base atteindra un état cohérent

Ainsi, une base NoSQL relâche certaines contraintes, telles que la synchronisation des réplicas, pour favoriser l'efficacité. **Théorème de Brewer dit "théorème de CAP"**

Le **théorème de CAP** dit :

Dans toute base de données, vous ne pouvez respecter au plus que 2 propriétés parmi la *cohérence*, la *disponibilité* et la *distribution*.

Cela s'illustre assez facilement avec les bases de données relationnelles, elles gèrent la cohérence et la disponibilité, mais pas la distribution.



CA - AP - CP

Prenons le couple **CA** (Consistency-Availability), il représente le fait que lors d'opérations concurrentes sur une même donnée, les requêtes L1 et L2 retournent la

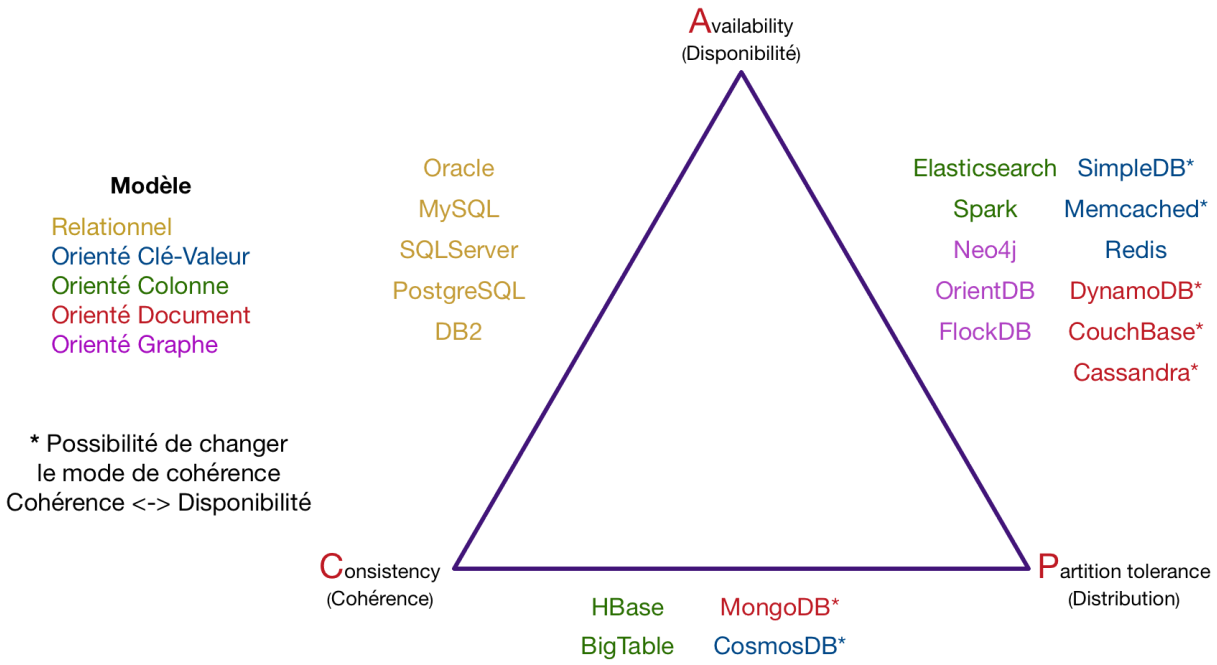
nouvelle version (v2) et sans délai d'attente. Cette combinaison n'est possible que dans le cadre de bases de données transactionnelles telles que les SGBDR.

Le couple **CP** (Consistency-Partition Tolerance) propose maintenant de distribuer les données sur plusieurs serveurs en garantissant la tolérance aux pannes (réplication). En même temps, il est nécessaire de vérifier la cohérence des données en garantissant la valeur retournée malgré des mises à jour concurrentielles. La gestion de cette cohérence nécessite un protocole de synchronisation des réplicas, introduisant des délais de latence dans les temps de réponse (L1 et L2 attendent la synchronisation pour voir v2). C'est le cas de la base NoSQL *MongoDB*.

Le couple **AP** (Availability-Partition Tolerance) à contrario s'intéresse à fournir un temps de réponse rapide tout en distribuant les données et les réplicas. De fait, les mises à jour sont asynchrones sur le réseau, et la donnée est "*Eventually Consistent*" (L1 voit la version v2, tandis que L2 voit la version v1).Ainsi, la cohérence des données est incompatible avec la disponibilité dans un contexte distribué comme les bases NoSQL.

Le triangle de CAP et les bases de données

Grâce à ce théorème de CAP, il est alors possible de classer toutes les bases de données en les plaçant sur le "triangle de CAP", tout en ajoutant des codes couleurs pour chaque modèle de stockage présenté dans le chapitre précédent.



Triangle de CAP

Nous pouvons constater que les bases de données relationnelles se retrouvent sur la face CA du triangle, combinant disponibilité et cohérence. Nous retrouvons bien

MongoDB pour le couple CP (cohérence et distribution) mais également les solutions orientées colonnes comme HBase ou BigTable.

Le couple AP (Disponibilité et distribution) regroupe le plus grand nombre de solutions NoSQL. En effet, la plupart cherchent les performances en relâchant volontairement la cohérence. Nous y retrouvons principalement des solutions orientées clé-valeur et graphes, mais également orientées documents (clé-valeur étendu).

Certaines solutions proposent également de modifier la politique de gestion de la concurrence (DynamoDB, CouchBase, Cassandra, MongoDB, CosmosDB...), dans ce cas, ils changent simplement de face sur ce triangle en passant de CP à AP.

L'avantage de ce triangle CAP est de fournir un critère de choix pour votre application. Vous pouvez ainsi vous reposer sur vos besoins en termes de fonctionnalité, de calculs et de cohérence.

Passez à l'échelle

Les origines de la distribution de données et le sharding

Les arbres

L'index le plus utilisé est l'arbre B (ou BTree), il adopte une structure arborescente pour chercher toute valeur indexée. Les feuilles de cet arbre contiennent des liens vers les pages contenant la valeur. Dans l'index d'un livre, les mots sont triés par ordre alphabétique. Dans un arbre, c'est la même chose en utilisant une recherche par dichotomie (plus petit/plus grand). Dans le cadre du NoSQL, l'arbre B va encore plus loin car les données sont triées : c'est un arbre B non-dense (ou **clustered index**). **Le hachage**

Le but d'une table de hachage est de placer les données par paquets, et à chaque donnée correspond un seul paquet de destination. Pour placer cette donnée, une **fonction de hachage** unique détermine le paquet. Pour reprendre notre analogie, nous parlerons cette fois de bibliothèque : les rangées A à C stockeront les livres dont le nom de l'auteur commence entre la lettre A et C. Pour retrouver un livre d'un auteur, il suffit d'aller dans la bonne rangée, même s'il faut ensuite examiner un certain nombre de livres pour le trouver.

La distribution

Les bases de données distribuées existent depuis de nombreuses années ; le but de la distribution est de soulager le serveur central en répartissant les données sur plusieurs serveurs. Ce serveur central s'occupe ainsi de répartir la charge (données

et requêtes), de fusionner le résultat et de gérer la cohérence des données. Cela vous fait penser au NoSQL ? Ne vous fiez pas aux apparences ! Les bases de données distribuées ne sont pas tolérantes aux pannes, la disponibilité est problématique en termes de synchronisation multi-serveurs, et il n'y a pas de techniques pour permettre l'élasticité de la base de données.

L'élasticité

L'*élasticité*,est la capacité du système à s'adapter automatiquement en fonction du nombre de serveurs qu'il dispose et de la quantité de données à répartir. Par exemple, vous avez 1To de données sur 1000 serveurs (1Go par serveur), lors d'un pic d'activité (soldes, période de Noël…), il serait utile de rajouter 1000 serveurs pour répartir la charge de calcul avec 500 Mo par serveur. L'élasticité va permettre de **répartir** uniformément les données sur les 2000 serveurs (déplacement de la moitié des données), et inversement lorsque le pic d'activité s'achève. De même, si le volume de données augmente et atteint 2To, l'élasticité garantira une répartition uniforme de 2Go par serveur.

Le sharding

Le *sharding* est une technique permettant de distribuer des **chunks** (morceaux de fichiers) sur un ensemble de serveurs, avec la capacité de gérer l'élasticité (serveurs/données) et la tolérance aux pannes. Trois familles de distribution pour le NoSQL existent : *HDFS* (basé sur la distribution), *le clustered index* (basé sur le BTree) et le *consistent hashing* (basé sur les tables de hachage). Regardons comment elles fonctionnent, elles nous permettront d'orienter nos choix d'architecture.

HDFS

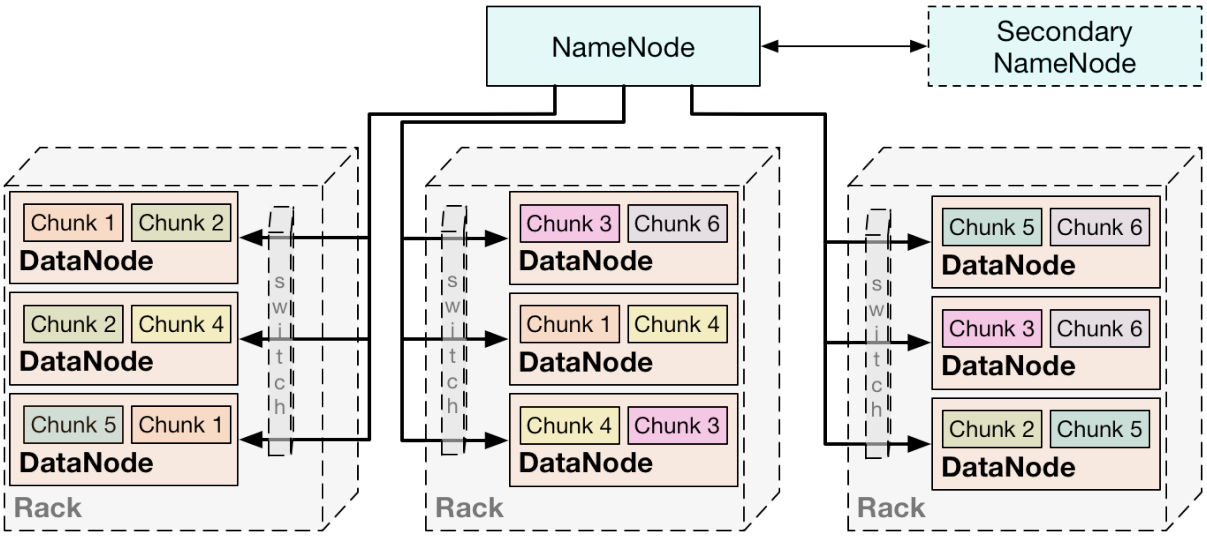
HDFS (*Hadoop Distributed File System*) est une technique de distribution de fichiers volumineux. Chaque fichier sera découpé en "chunk" de 64Mo pour être distribué sur le réseau. Chaque serveur de ce réseau est un *datanode* contenant plusieurs chunks. La répartition de ces chunks est définie par le serveur central, le *namenode*.

Chaque chunk est répliqué sur 3 serveurs distincts ; de fait, si un serveur tombe en panne, la donnée peut tout de même être récupérée. Nous pouvons constater également que le **namenode** est lui-même répliqué avec un s*econdary namenode*. Celui-ci permet au serveur central de redémarrer rapidement en cas de panne.

L'autre différence est qu'aucun traitement n'est effectué sur le namenode. Son rôle est de router et d'administrer les datanodes. Les traitements se font donc sur les datanodes, ainsi que la répartition des résultats (produit par Map/Reduce).

La distribution des chunks est maîtrisée par le namenode, répartissant ceux-ci en fonction de la structure physique du réseau, minimisant ainsi les problèmes de

pannes (serveurs sur un même switch ou rack). On dit que le namenode est **rack-aware**.



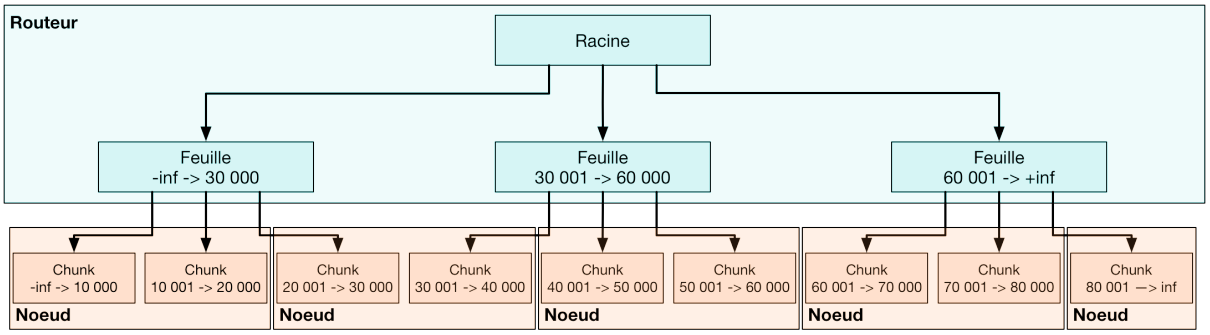
Sharding sous HDFS

L'**élasticité** se fait naturellement. Lorsque l'on rajoute un datanode, il récupère des chunks de ses voisins pour répartir la charge. Lorsqu'un datanode disparaît, les chunks associés sont récupérés sur les replicas pour être répartis sur d'autres nœuds. Toutefois, la synchronisation des chunks en fait une solution sensible aux mises à jour.

L'avantage de cette solution est une forte puissance de calcul et une bonne tolérance aux pannes. On retrouve de nombreuses solutions NoSQL reposant sur cette architecture, notamment celles orientées colonnes : *HBase*, *PigLatin*, *Spark*.

Un arbre distribué

La seconde famille de distribution de données repose sur le traditionnel BTree non-dense (ou **clustered index**) : il s'agit de l'arbre dont les données sont triées. Un serveur central s'occupe de l'arborescence de cet arbre, et les feuilles (les données) sont prises en charge par les nœuds du cluster.



Sharding arborescent

Le serveur central agit comme **"routeur"** en donnant accès au serveur contenant la donnée requise. La gestion de l'arborescence lui permet de connaître la répartition des valeurs par nœud et, de fait, d'agir sur l'élasticité en modifiant les bornes de chaque nœud. Ceci permet de préserver en permanence le tri des données. Afin de garantir les pannes, le routeur est *répliqué* et oblige la synchronisation de l'arborescence.

Les nœuds contenant les données s'occupent des traitements (Map/Reduce), mais gèrent également la *réplication* des données. Contrairement à HDFS, la réplication est maintenue par le nœud lui-même, à cause de l'arborescence imposée par l'arbre. Nous pourrons voir dans le chapitre sur **MongoDB** la technique du *ReplicaSet* permettant d'effectuer cette réplication.

Les avantages de cette technique sont :

1. rassembler les données ayant des valeurs similaires (tri), et de fait toute opération sur ces données rassemblée (regroupement/reduce)
2. faciliter la gestion de la cohérence des données en verrouillant au niveau du nœud l'information et de synchroniser lui-même ses données avec ses replicas.

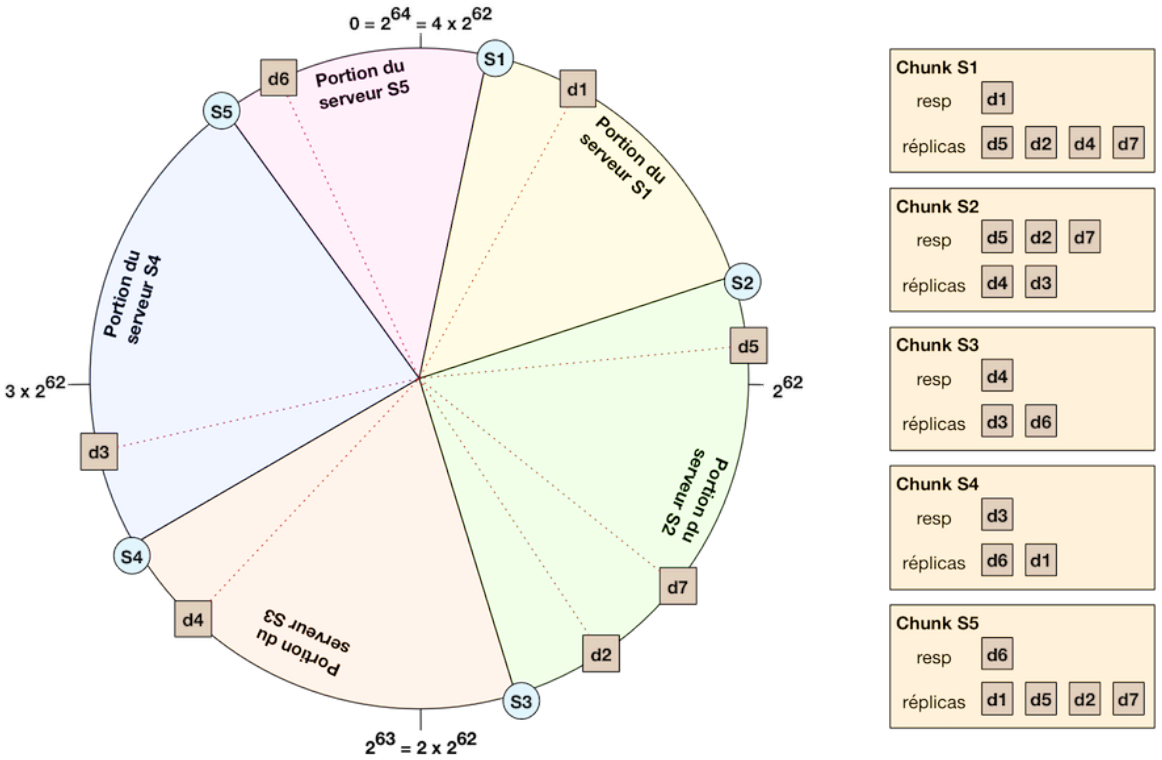
Une table de hachage distribuée (DHT)

Avec la dernière famille, basée sur les tables de hachage, on distribue l'intégralité des informations dont on dispose : à la fois la donnée et la table de hachage. C'est ce que l'on appelle le **Consistent Hashing**. La particularité de cette technique est que chaque nœud est à la fois client et serveur. Imaginez un anneau contenant

2^{64} serveurs (soit $1{,}8\times10^{19}$, la *fonction de hachage* place chaque donnée sur cet *anneau*, considéré comme un angle dans un cercle trigonométrique. Un nœud s'occupe de toutes les données dont l'angle est compris entre lui-même et le nœud suivant (*portion du cercle*), un peu comme une part de gâteau.

Bien sûr, vous n'avez pas à allouer 2^{64} pour faire fonctionner ce réseau. Cet anneau est virtuel et chaque serveur physique est placé sur cet anneau et prend en charge la portion du cercle allant jusqu'au prochain serveur physique. Comme on peut le

voir sur la figure, les 5 serveurs se partagent le gâteau. Les portions ne sont pas forcément de même taille mais doivent équilibrer une certaine charge pour constituer les *chunks*. Les données de ces chunks sont placées grâce à sa valeur de hachage (fonction donnant un nombre entre 0 et 2^{64}, forcément). Cette valeur de hachage peut être considérée comme un angle dans ce cercle, et le nœud ayant la portion contenant cet angle est responsable de la donnée.



Sharding avec une DHT

L'*élasticité* est simple à obtenir : il suffit d'allouer un serveur à une place de l'anneau ayant une trop grande quantité de données. Dans notre exemple, il suffirait d'ajouter un serveur entre S2 et S3 (en gros entre d5 et d7 ou d2 et d7) pour équilibrer la charge. En ce qui concerne la réplication, chaque donnée est répliquée sur trois serveurs physiques contigus sur l'anneau. Notre exemple montre pour chaque chunk les réplicas associés (d4 dont est responsable S3, est répliqué sur les serveurs S2 et S1). Ainsi, lorsqu'un nœud disparaît, le nœud précédent devient le responsable (déjà répliqué) ; il suffit alors de répliquer la donnée une nouvelle fois.

Pour se déplacer dans l'anneau, on ne va clairement pas faire des sauts de serveur en serveur, ce serait interminable ! On va effectuer des **sauts** plus grands, mais dépendant de l'endroit où est située la donnée recherchée. Dans la figure ci-dessus, supposons que le serveur S1 cherche la donnée d3. S1 va procéder par **dichotomie** par rapport au cercle, la donnée est-elle plus loin que le côté opposé du cercle ? En effet, donc on va dans le nœud qui a la charge de cette portion : S3. Du coup, S3 fait lui-même la recherche. Est-ce plus de la moitié ? non. Plus que le quart ? non plus.

Plus que le ⅛ ? oui ! On va sur le serveur en question : S4. La donnée est sur S4, on est bon ! Cette dichotomie permet de garantir un temps de réponse acceptable pour toute requête.

Les avantages pour cette technique sont :

1. une architecture totalement distribuée capable de s'**auto-gérer**
2. une **élasticité** simplifiée, même s'il est difficile d'identifier les zones en surcharge (pas de structure de contrôle).

Par contre, cette technique ne peut bénéficier des avantages du HDFS, comme la tolérance à différents types de pannes (c'est un réseau logique et non physique) ou la distribution du BTree (pas de tri).

Choisissez votre base de données NoSQL

Quels sont les critères de choix d'une BDD NoSQL ?

Le choix de votre système est primordial pour la suite, et il ne faut pas le faire à la légère. Je vous propose une approche pour prendre une décision Mais tout d'abord, il nous faut des critères de choix.

Le coût

Le **coût** de la solution est un facteur important . Il est bon de consulter l'offre et de voir si une version professionnelle est disponible (payante), les services associés (facilité d'administration/exploitation/déploiement). Le coût d'infrastructure est similaire quelle que soit la solution car dépendant de vos besoins et de l'implémentation des algorithmes que vous aurez réalisés. Toutefois, les offres « Cloud » ne sont pas forcément toutes adaptées à votre choix. **La cohérence des données**

La **cohérence** des données est un critère fondamental pour des applications reposant sur des données « fraiches » ou des applications reposant sur des décisions critiques.

La disponibilité du système

À opposer à la **disponibilité** grâce au théorème de CAP, nous pouvons déduire que ces deux critères ne sont pas compatibles. Attention, les temps de latence restent faibles (quelques centaines de millisecondes), mais si cela reste crucial, il ne faut pas hésiter. Toutefois, cela ne concerne que les mises à jour de données déjà existantes, pas les insertions.

Le langage d'interrogation

La richesse du **langage** permet également de manipuler simplement la base NoSQL. Il facilite la conduite du changement des développeurs dans le processus d'intégration de la solution. Plus le langage est riche, plus vous serez à même d'exprimer vos besoins. Toutefois, une formation est souvent nécessaire pour comprendre les subtilités du langage.

Les fonctionnalités

La **fonctionnalité** recherchée est souvent un critère clé dans le choix. Le but est de répondre à une demande : data mining, recherche textuelle, temps-réel, graphe, outil compatible…

Spécifiez vos besoins

Maintenant que nous avons défini nos critères de choix, il faut prendre une décision. Pour cela, prenez le panel de solutions NoSQL candidates, mettez-le dans une **matrice multicritères**. Cette matrice donne en abscisse les critères de choix, et en ordonnée les solutions proposées.

Il faut tout d'abord donner un poids à chacun de ces critères. Nous en avons choisi 5, il suffit maintenant de répartir pour chaque critère une valeur entre 0 et 5. La seule contrainte est que le somme de ces poids soit égale à 5. Un poids de 0 signifie que ce critère n'a aucune importante, et une note supérieure à 2,5 qu'il est primordiale (car il représente à lui seul la moitié des critères).

	Poids	MongoDB	Cassandra	ElasticSearch
Coût	0,5	4	4	4
Cohérence	1	4	2	2
Disponibilité	0,5	2	4	3
Langage	1	4	2	3
Fonctionnalité : Symfony	2	4	2	4

Total	***5***	***19***	***12***	***16,5***

Il reste maintenant à déterminer pour chaque solution s'il répond ou non à la demande en lui attribuant une **note** (et non un poids). Cette note va de 0 à 4, de manière qualitative (insatisfaisant à satisfaisant) ou quantitative (temps de réponse, volume maximum).

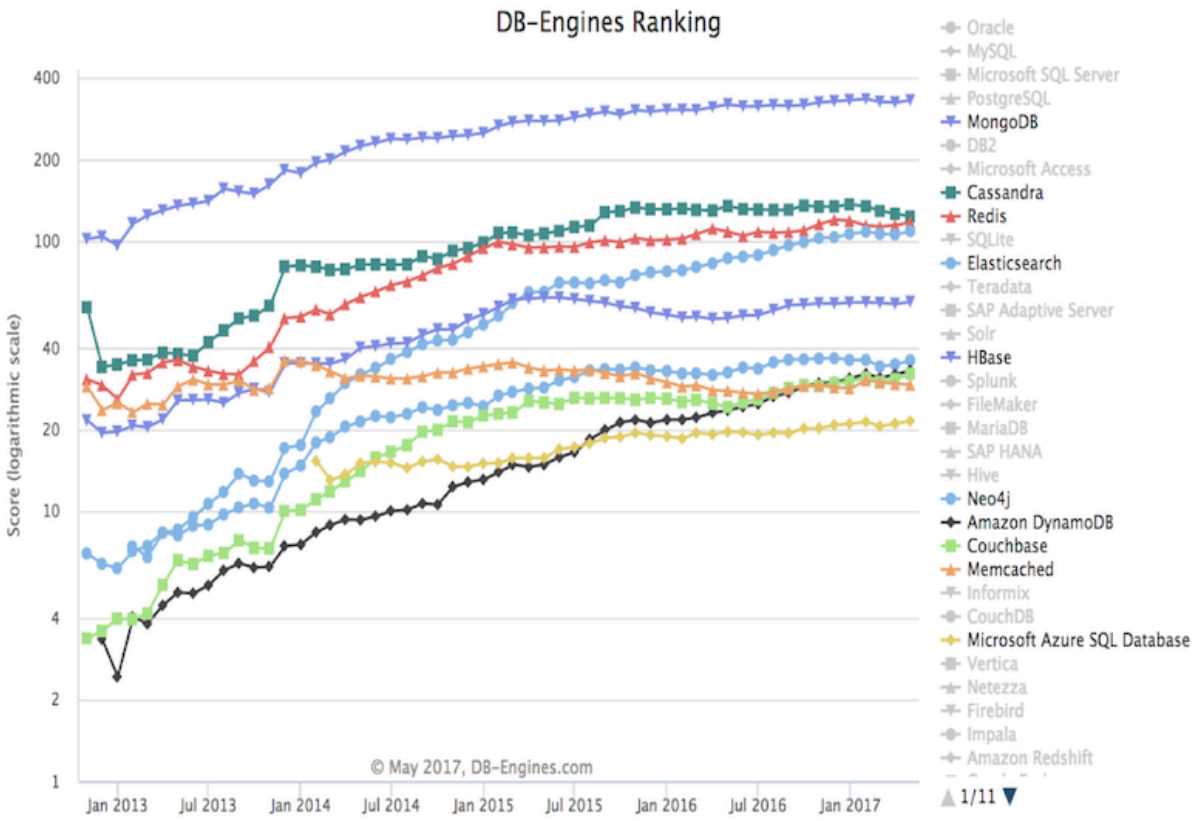
Pour prendre votre décision, il suffit tout naturellement de multiplier vos poids avec les notes. La note finale est sur 20 (5 critères notés maximum sur 4, ce qui fait 4x5 = 20). La somme la plus grande correspond à la solution donnant le meilleur compromis sur tous vos critères.

Le tableau donné en exemple parle de lui-même. La pondération d'un projet utilisant le framework *Symfony*, avec gestion de la cohérence et un langage riche oriente le choix vers *MongoDB*. On peut être surpris par le résultat de *Cassandra*, mais le fait qu'il ne soit pas intégré nativement à Symfony et son langage trop réduit par rapport à SQL en fait une solution non envisageable. De son côté, *Elasticsearch* est une bonne solution, mais est rejetée pour sa gestion de la cohérence et son langage trop complexe.

En manque d'inspiration ? Allez sur DBEngines

Allez, un petit bonus pour la fin, toujours dans l'idée de permettre de faire votre choix. Le site web DB-Engines.org donne de nombreuses caractéristiques sur toutes les bases de données référencées sur le Web. Non seulement il donne les caractéristiques pour chaque BDD, mais il évalue également son impact sur la toile.

En répertoriant chaque ticket sur *stackoverflow* (bug, solution, code, conseil…), les blogs et tutoriels, leur utilisation par les entreprises (spécifié dans les rapports d'architecture), *DB-Engines* produit un score pour chaque solution. Le but est de connaître les tendances, aussi bien pour les développeurs que les entreprises. Toutefois, cela n'est qu'un prisme sur le réel, basé uniquement sur les informations présentes sur internet. Mais cela donne déjà un avis et permet de se poser des questions.



DB-Engines - Classement des bases NoSQL

Le graphique ci-dessus représente les tendances de janvier 2013 à juin 2017. J'ai volontairement fait un focus sur les solutions NoSQL citées dans ce cours, mais on peut voir aussi (en gris) que les bases de données relationnelles, distribuées et orientées colonnes (sans être NoSQL) sont également prises en compte.

Commentaires

MongoDB reste une solution fortement utilisée par les développeurs. Ce qui en fait sa popularité est son langage puissant et le fait qu'il soit facilement intégrable dans toute application gérant des documents/objets.